

20260913外发版-基于weakness和skill 的数据构造方案

注意：需要和“外发版-评测题包交付规范（rewardkit）20262913”组合使用。

1. 项目背景与使用范围

- 本文件定义该RL从0到1标注生产下数据的规格、格式、质量标准与验收流程。
- 适用对象：提供标注数据的所有数据生产流程。
- 面向对象：供应商、内外部专家、标注质检平台。
- 写给领域专家：
 - 作为对应办公场景的专家，你熟悉该领域的任务要求，熟悉该领域办公目标实现的产物标准。
 - 你的工作是根据自己擅长的领域，选择合适难度的任务，给出能对应办公场景、具备真实业务价值的**任务描述**，并且给出恰当的**参考文件**，以及对应该任务的**参考答案**。同时你需要针对该任务，给出一套能衡量任务做得好坏的**打分项**。
 - 我们会根据你给的打分项
 - ① 对你提交的参考答案打分，看得分是否足够高（正确率>0.85）；
 - ② 用3个模型执行你给出的任务，并用你给出的打分项打分，看难度是否足够高。
 - 此外，人工质检会重点考察题目真实性、参考答案合理性、打分项设计合理性、是否hack等异常作弊情况，一经发现，直接打回。

2. 难度分级

- 按三模型平均正确率划分难度：只要平均分低于 70% 即算难度达标；达标后再按下表归入 A1 基础 / A2 进阶 / A3 高难三档（区间不重叠）。
- 三模型平均正确率：
 - 单条：Claude code框架下，跑 gpt-5.6-sol、claude-opus-4-8、qwen3.8-max0902三个模型，。所有题目满分归一化为 1.0 的情况下，对于GPT-5.6 sol / Opus 4.8 / Qwen3.8-max 0902 三个模型，裁判模型为qwen3.7 plus，每种模型都跑1次求平均分，均分需要 < 0.7 ；且至少有一个模型有得分（避免全 0 的"死题"）。

难度分级定义			
级别	产量占比	数量	平均正确率

A1 基础	20%	200（单个领域50题）	60%≤得分<70%
A2 进阶	60%	600（单个领域150题）	50%≤得分<60%
A3 高难	20%	200（单个领域50题）	得分<50%

首批构造的数据需要覆盖2个方面：

领域覆盖：通用办公GO、金融FIN、法律LAW、医疗MED、数据分析DA、游戏GA、设计DE，数据需要平均覆盖不同领域，同时不同领域下的数据应该均匀分布，不接受一个知识点/workflow出题大于等于3道。且数据分布符合后文专项数据和weakness数据的要求。

数据总量：2000条（基于skill/tool/workflow等内容专项数据1000条，weakness数据1000条）

跑分产物及轨迹要求：在提交题目文件时，请同步提供模型产物和跑分轨迹，用于验证难度要求。

一、专项数据：基于skill/tool/workflow等内容专项数据

更关注某一能力方向的系统性和完整覆盖，例如 Tools / Skills / Workflow 专项需要从能力维度出发，尽可能全面覆盖不同场景、难度、环境和任务形态，目标是整体提升该能力

一级	二级	数量
Skills 数据	Skill Discovery	200
	Skill Generation / Editing	200
	Skill Dependency	200
Workflow Execution	Dependency-aware Workflow	200
	Subagent Workflow	200

二、Weakness-driven 数据：

需要更聚焦已经观察到的具体模型缺陷，从明确的 Bad Pattern / Trigger 出发，针对性构造能够稳定复现并训练该问题的场景。

数量：1000条，要求覆盖14种weakness，以及数据复杂性覆盖C1-C5

- 数据需要在元数据中标注数据主要作用于哪个weakness，必须覆盖1个以上weakness。
- 1000 条数据须覆盖全部 14 种 weakness。计数规则：每条数据在 weakness_tag 中标注 ≥ 1 个；一条数据标注多个时，对其标注的每一种各计 1 次覆盖。每种 weakness 至少被 50 条数据覆盖，且不超过 250 条。标注须与题目实际埋入的 trigger 一一对应：标了某个 weakness，题面和 environment 中必须存在能稳定诱发该 weakness 的设计点。
- C1-C5 分级参考如下

等级	文件数	Requirement 数/约束	输出产物数（与文件数不绑定，主要做参考，核心是需要复杂任务的任务）	Evidence Chain	执行步数 / 子任务数	工具种类数	领域专业深度（参考）	题量	占比
C1	2	3	1-2	1-hop	1-3 / 1	1	通用常识	120	12%
C2	5	6	2-3	2-hop	4-7 / 2-3	2	基础专业	250	25%
C3	10	10	3-4	3-hop	8-15 / 4-6	3	中级业务	350	35%
C4	25	15	4-6	4-hop	16-30 / 7-10	4	高级专业	180	18%
C5	50+	20	4-8（如果是一个大型项目，如做整个法律尽职调查，产出物页可以为1份。）	5-hop	30+ / 10+	5+	专家级	100	10%

各等级画像与典型场景

等级	画像	典型真实生产力场景
C1	单文件、单输出、单步完成，无需专业知识	从一份 Excel 中统计销售额并生成一句话结论
C2	多文件简单对照，2 种工具，基础专业门槛	对比两份报价单，输出差异表 + 推荐方案
C3	标准项目规模，中等推理链，3 种能力组合	根据销售数据、客户反馈、产品资料生成分析报告 + 图表 + 建议清单
C4	复杂项目级任务，长依赖链，需要高级领域知识	基于季度财报、行业研报、内部纪要撰写投资 memo + 财务模型 + 风险清单
C5	大型项目/知识库，多产物完整交付，专家级判断	完成并购项目的 CIM 分析、估值模型、尽调问题清单

指标说明

维度	含义	计量方式
文件数	模型可读取/处理的输入文件数量	不含重复文件、临时文件
Requirement 数	显式或隐式需满足的条件/约束条数	含功能、格式、引用、质量约束
输出产物数	最终需交付的文件/片段/结果数量	如代码文件、表格、报告段落、邮件等
Evidence chain 深度	完成答案所需的最长跨文件推理步数	1-hop = 单文件直接可得
执行步数	模型需调用的原子操作/工具调用次数	工具调用次数
工具种类数	指不同类型的能力/工具，如：文本阅读、搜索、代码执行、计算、写作、绘图、表格处理等。同一类工具的多次调用只算 1 种。	工具种类数量

Weakness-driven 数据构造方案：

- 数据需要在元数据中标注数据主要作用于哪个weakness，必须覆盖1个以上weakness

- weakness_tag为b列

	A	B	C	
1	优先级	Weakness	典型行为	
2	W1	长文档下的硬约束遵循较差	模型不是完全没看到要求，而是随着内容生成变长，部分 hard constraint 从 working state 中丢失；尤其是需要真实 artifact operation 才能满足的约束，例如 section break、页码域、字数上限、公式保留，而不是在正文中“写到了”就算完成。	PA-T1-001 5字，Qwen 最终
3	W2	关键冲突下自行选边 / 修改输入，而不是模糊澄清或是说明具体冲突情况	Workspace / Context 中存在互相矛盾的关键信息，模型甚至已经发现冲突，但会自行选择其中一个版本继续，极端情况下直接“修正”输入。报告中血钾 8.2 mmol/L vs “持续低钾”、材料用量 100ml vs 120ml 都稳定复现了该问题。	医疗冲突 Case mmol/L。高钾：定哪一条是真的
4	W3	阻塞状态的识别与恢复能力较差	模型在任务执行过程中遇到关键资源缺失、权限受限、外部依赖失败、输入损坏或必要信息不足等 Blocking State 时，无法稳定判断 典型错误主要有两类： 过早放弃 / 不懂变通：原路径失败后直接停下或把任务推回用户，明明存在 retry、替代工具、备用数据源等可恢复路径； 强行闭环 / 伪完成：关键阻塞尚未解除，却使用假数据、placeholder、自行估值、未经授权的替代方案继续执行，并最终宣称任务完成	case1 : case2: 某相
5	W4	多要求 / 多交付物下缺少	简单任务基本能完成，但当显式要求项、文件数、sheet / section / field / 时，模型容易“完成主文	hard_194 : PPT 要求同时交付

Qwen 在要求项更多、交付物更多、跨应用更多的 hard query 上，思考和工具调用没有随难度增加，甚至有所下降, 明显弱于其他模型；而 Opus 会相应增加执行与验收投入。这个现象更适合作为**所有定向 weakness 数据的 difficulty-control 维度**。

基于skills/workflow的scaling的专项数据

1. 背景 & 出发点

目前 Cowork RL 数据中，大部分任务仍运行在相对简单的环境中：

- 以 filesystem / shell / 少量原生工具为主；
- Tool schema、可用能力和调用方式相对固定；
- Skill 即使存在，也经常只是辅助信息，并非完成任务所必需；
- Workflow 多数是线性的「读取 → 处理 → 输出」，较少包含真实业务中的状态变化、异步依赖、异常恢复、跨工具协同等复杂过程。

下一阶段希望数据建设从单纯扩充 Task Distribution，进一步转向扩充 Environment Distribution。

2. 整体数据目标

本专项主要覆盖以下几类能力：

Skill Dependency

任务必须真正依赖 Skill 中的特有信息、规则或 SOP 才能稳定完成。

skill能够包含一些特有信息或行为规则，避免 Skill 成为一个可有可无的 contextual document

Workflow Execution

构造真实存在：多步骤依赖；多工具协同；branch；async；blocking state；subagent；的长链路任务。（最好是有涉及subagent的，例如在codex下运行该任务会触发subagent）

MCP Scaling（低优）

扩大模型在非原生工具、陌生 Tool Schema 和不同 Tool 组合下的泛化能力。可主要通过 MCP

数据需要基于各 Agent Framework 可接入的 Plugin / MCP 能力构造任务，并在每条任务中明确提供当前 Environment 下模型可使用的工具集合 / MCP。工具可以是框架已有能力，也可以是动态注入的自定义工具（在题目里显式说明）。

3. Skills 数据需求

Skill Discovery

提供多个 Skill, 要求模型自行判断：是否需要 Skill，以及应该使用哪个 Skill。

Skill Dependency

构造：

没有 Skill 中的特殊知识 / 行为规则，模型很难正确完成任务。

推荐构造方式

类型一：特殊业务规则

Skill 明确规定：当两个 authoritative source 的关键字段冲突时，禁止自行选择，必须暂停并向用户确认。

Task 中主动埋入这种冲突。模型如果没有真正遵循 Skill：最终一定会进入错误 branch。

类型二：环境特化 SOP

例如 Skill 定义某企业内部审批流程：Draft → Compliance Review → Manager Approval → Publish

Task 要求完成发布。如果模型直接：Draft → Publish。最终 Workflow State 必须失败。

类型三：特殊计算 / 判定规则

Skill 中包含：特殊阈值；内部映射规则；自定义字段定义；业务优先级；fallback 顺序。要求这些内容直接影响 final answer。

Skill Generation / Editing

可以增加一批：创建 Skill；修改已有 Skill；从 SOP 抽取 Skill；Skill migration；Skill package repair之类的任务

4. Workflow 数据需求

Dependency-aware Workflow

任务中存在明确 dependency：

A → B
A → C
B + C → D
D → Final

要求模型：

- 正确拆解 dependency；
 - 维护哪些任务已完成；
 - 避免提前执行依赖未满足的步骤；
 - Final 前确认 dependency 已全部完成。
-

Subagent Workflow

构造必须依赖多个 Subagent Result 才能完成 Final 的任务。

数据交付 Metadata

建议每条数据额外维护以下字段：



 复制代码

```
skill_set = ["approval-workflow"] # 环境内安装的功能型 skill (= environment/skills/ 下的目录名)
```

```
task_complexity = "C3" # C1-C5, 取值
口径见正文分级表
expected_skill_dependencies= ["approval-workflow"] # 完成任务真正必须依赖的 skill（子集，可为空表示 Skill Discovery 干扰项）
weakness_for = ["107 流程步骤", "112 约束满足"] # 覆盖 1 个
```

字段	类型	取值写法 / 约束
skill_set	array	环境内安装的功能型 skill 目录名，须与 environment/skill_ls/<name>/ 逐一对应；可含干扰项（Skill Discovery 场景）
task_complexity	enum	C1 – C5，按正文分级表的文件数/requirement/evidence-hop 等硬指标判定
expected_skill_dependencies	array	skill_set 的子集：不依赖它任务必失败的 skill。 Skill Discovery 任务 里这一项应严格小于 skill_set（存在故意的无关 skill）
weakness_tag	array	覆盖 ≥1 个；

本专项构造的数据需要遵循以下完整task.toml参考：

```
task.toml
```

复制代码

```
schema_version = "1.4" # 照抄

# 格式: /app/output/{你的交付物名称}
artifacts = [
    "/app/output/FIN-T2-001_财务分析报告.xlsx",
    "/app/output/FIN-T2-001_毛利率趋势.png",
    "/logs/artifacts/output",
]

[task]
name = "work/fin1-001" # 任务名称
version = "1.0.0" # 任务版本
description = "基于两期年报与交易流水产出财务分析报告" # 任务描述

[metadata]
task_id = "FIN1-skill-DEP-001" # 题目编号
author_organization = "<供应商名>"
category = "skill-dependency"
```

```

domain = "金融" # 所属领域：通用办公/金融/医疗/法律
domain_l2 = "投资银行" # 所属领域二级标签（对齐知识体系表）
capabilities = "财务比率计算、估值口径" # 评测的能力：该任务能评测的具体专业能力
difficulty = "A2" # 难度等级：A1易/A2中/A3难
vl_dependency = "否" # 是否依赖VL（视觉-语言能力）：是/否
source_note = "任务来源、真实性保障、真实场景中的交付物是什么" # 任务说明
tools = "Excel/Python" # 所需工具（办公工具/数据库，Word/Excel/PPT 等）
domain_knowledge = "再融资定增的财务分析口径与可比公司估值方法" # 领域知识依赖（选填）
task_complexity = "C2" #取值C1-C5
weakness_tag = ["W07-流程跳步", "W12-约束遵循"] # 覆盖≥1个，按算法词表填写，至少一项
environment_template = "<平台已确认的环境模板名>"
tool_set = ["filesystem", "shell", "python"] # 按实际可用能力填写
skill_set = [] # 本示例未提供技能；有技能时填写目录名
expected_tool_dependencies = ["filesystem", "shell", "python"] # 按实际必要能力填写
expected_skill_dependencies = [] # 必须是 skill_set 的子集
tags = ["finance", "office", "A2", "skill-dependency", "sop-enforcement", "workflow", "docx", "llm-judge"]
# 逐题填写任务关键词

[[metadata.deliverables]] # 交付物一
path = "FIN-T2-001_财务分析报告.xlsx" # 交付物一的路径
required = true
desc = "主交付物：财务分析报告" # 交付物一的描述

[[metadata.deliverables]] # 见交付物一的说明，如有多个交付物，必须依次写明
path = "FIN-T2-001_毛利率趋势.png"
required = true
desc = "毛利率趋势折线图"

[agent]
timeout_sec = 72000.0 # 缺省 72000;

[verifier]
timeout_sec = 18000.0
user = "root"

[verifier.env]
# Judge 相关变量统一 JUDGE_ 前缀，只写在 [verifier.env]、严禁写入 [environment.env]。
# 必选项（运行环境注入，AP 必给）；可选项一律用 ${VAR:-默认值} 语法给默认值。

```

```

JUDGE_API_KEY = "${JUDGE_API_KEY:-}" # 必选: judge 网关 key
JUDGE_BASE_URL = "${JUDGE_BASE_URL:-}" # 必选: judge 网关地址 (可
带 /v1, test.sh 负责剥)
JUDGE_MODEL = "${JUDGE_MODEL:-qwen3.7-plus}" # 可选: 裁判模型 (经
REWARDKIT_MODEL 运行时生效)
JUDGE_PROVIDER = "${JUDGE_PROVIDER:-anthropic}" # 可选: 信息性声明
JUDGE_API_PROTOCOL = "${JUDGE_API_PROTOCOL:-anthropic}" # 可选: openai
时降级为 LLM judge (见 test.sh)
# 兼容旧环境的兜底注入
EVAL_API_KEY = "${EVAL_API_KEY:-}"
EVAL_API_BASE = "${EVAL_API_BASE:-}"
LITELLM_DROP_PARAMS = "true"

[environment]
os = "linux"
build_timeout_sec = 18000.0
network_mode = "public" # claude-code 框架下不要写 no-network (见
3.2 警示)
cpus = 2
memory_mb = 8192
storage_mb = 30720 # 照抄

```

Skill形态说明

- 当前可以主要通过写入到skills文件夹下的形式

功能型Skill的标准目录结构

复制代码

```

AQ-002/
├─ task.toml # 任务声明
├─ instruction.md # 显式说明"通过 contract-redliner
skill 完成, 禁止其它方式改文档"
├─ environment/
│   └─ Dockerfile # COPY skills/ /skills/ ← skill 在这
里被装进环境
│   └─ skills/
│       └─ contract-redliner/
│           └─ SKILL.md # Anthropic Skill 格式:
frontmatter(name/description/compatibility)+正文SOP
│           └─ scripts/ # read_document.py /
propose_edits.py / add_comment.py ...

```

```

|   |   └─ references/           # anchors.md 等被 SKILL.md 引用的补充规
则
|   └─ app/                     # 任务输入: contract.docx + grounding
材料
└─ tests/
   └─ test.sh → run_verifier.py # 判分入口
   └─ rubrics.json              # 判据
└─ solution/
   └─ solve.sh

```



AQ-001.zip
153.4KB

SKILL.md 的写法参考:



复制代码

```

---
name: contract-redliner
description: Redline Word (.docx) contracts with native tracked changes
and margin comments ... Use when asked to redline, mark up, revise a
.docx legal document – every edit must be a real tracked change applied
via the bundled scripts, never by rewriting the file.
compatibility: Requires Python 3.10+ with python-docx==1.2.0, docx-
revisions, lxml.
---

```

Contract Redliner

(正文: 分步 SOP、脚本用法、失败恢复、"绝不能用别的方式改文档"的硬规则——
这些"特有行为规则/阈值/映射"就是让 Skill 不可省略的关键

注意 [instruction.md](#) 里需要显式强调下要使用该skill:

Every edit and every comment goes through the **contract-redliner** skill installed in this environment — never write `[ADD:…]` / `[DELETE:…]` markup, and never modify the document by any other means.

task.toml参考



复制代码

```

schema_version = "1.4" # 照抄

# 格式: /app/output/{你的交付物名称}
artifacts = [
  "/app/output/FIN-T2-001_财务分析报告.xlsx",

```

```

"/app/output/FIN-T2-001_毛利率趋势.png",
"/logs/artifacts/output",
]

[task]
name = "work/fin1-001"           # 任务名称
version = "1.0.0"                # 任务版本
description = "基于两期年报与交易流水产出财务分析报告" # 任务描述

[metadata]
task_id = "FIN1-skill-DEP-001"   # 题目编号
author_organization = "<供应商名>"
category = "skill-dependency"
domain = "金融"                  # 所属领域：通用办公/金融/医疗/法律
domain_l2 = "投资银行"           # 所属领域二级标签（对齐知识体系表）
capabilities = "财务比率计算、估值口径" # 评测的能力：该任务能评测的具体专业能力
difficulty = "A2"                # 难度等级：A1易/A2中/A3难
vl_dependency = "否"             # 是否依赖VL（视觉-语言能力）：是/否
source_note = "任务来源、真实性保障、真实场景中的交付物是什么" # 任务说明
tools = "Excel/Python"          # 所需工具（办公工具/数据库，Word/Excel/PPT 等）
domain_knowledge = "再融资定增的财务分析口径与可比公司估值方法" # 领域知识依赖（选填）
task_complexity = "C2"           #取值C1-C5
weakness_tag = ["W07-流程跳步", "W12-约束遵循"] # 覆盖≥1个，按算法词表填写，至少一项
environment_template = "<平台已确认的环境模板名>"
tool_set = ["filesystem", "shell", "python"] # 按实际可用能力填写
skill_set = []                     # 本示例未提供技能；有技能时填写目录名
expected_tool_dependencies = ["filesystem", "shell", "python"] # 按实际必要能力填写
expected_skill_dependencies = []    # 必须是 skill_set 的子集
tags = ["finance", "office", "A2", "skill-dependency", "sop-enforcement", "workflow", "docx", "llm-judge"]
# 逐题填写任务关键词

[[metadata.deliverables]] # 交付物一
path = "FIN-T2-001_财务分析报告.xlsx" # 交付物一的路径
required = true
desc = "主交付物：财务分析报告" # 交付物一的描述

[[metadata.deliverables]] # 见交付物一的说明，如有多个交付物，必须依次写明
path = "FIN-T2-001_毛利率趋势.png"

```

```

required = true
desc = "毛利率趋势折线图"

[agent]
timeout_sec = 72000.0          # 缺省 72000;

[verifier]
timeout_sec = 18000.0
user = "root"

[verifier.env]
# Judge 相关变量统一 JUDGE_ 前缀，只写在 [verifier.env]、严禁写入
[environment.env]。
# 必选项（运行环境注入，AP 必给）；可选项一律用 ${VAR:-默认值} 语法给默认值。
JUDGE_API_KEY = "${JUDGE_API_KEY:-}"          # 必选: judge 网关 key
JUDGE_BASE_URL = "${JUDGE_BASE_URL:-}"        # 必选: judge 网关地址（可
带 /v1, test.sh 负责剥）
JUDGE_MODEL = "${JUDGE_MODEL:-qwen3.7-plus}"  # 可选: 裁判模型（经
REWARDKIT_MODEL 运行时生效）
JUDGE_PROVIDER = "${JUDGE_PROVIDER:-anthropic}" # 可选: 信息性声明
JUDGE_API_PROTOCOL = "${JUDGE_API_PROTOCOL:-anthropic}" # 可选: openai
时降级为 LLM judge（见 test.sh）
# 兼容旧环境的兜底注入
EVAL_API_KEY = "${EVAL_API_KEY:-}"
EVAL_API_BASE = "${EVAL_API_BASE:-}"
LITELLM_DROP_PARAMS = "true"

[environment]
os = "linux"
build_timeout_sec = 18000.0
network_mode = "public"          # claude-code 框架下不要写 no-network（见
3.2 警示）
cpus = 2
memory_mb = 8192
storage_mb = 30720              # 照抄

```